

ENTREGA 1

ENGENHARIA DE SOFTWARE

DESCRIÇÃO DO PROJETO

Nome do projeto: **ESC**

Nome das equipes:

- Eric De Lucas Silva
- Stephanie Macedo da Silva
- En Hsiang Chien

Nome do orientador:

- Luis Fernando dos Santos Pires

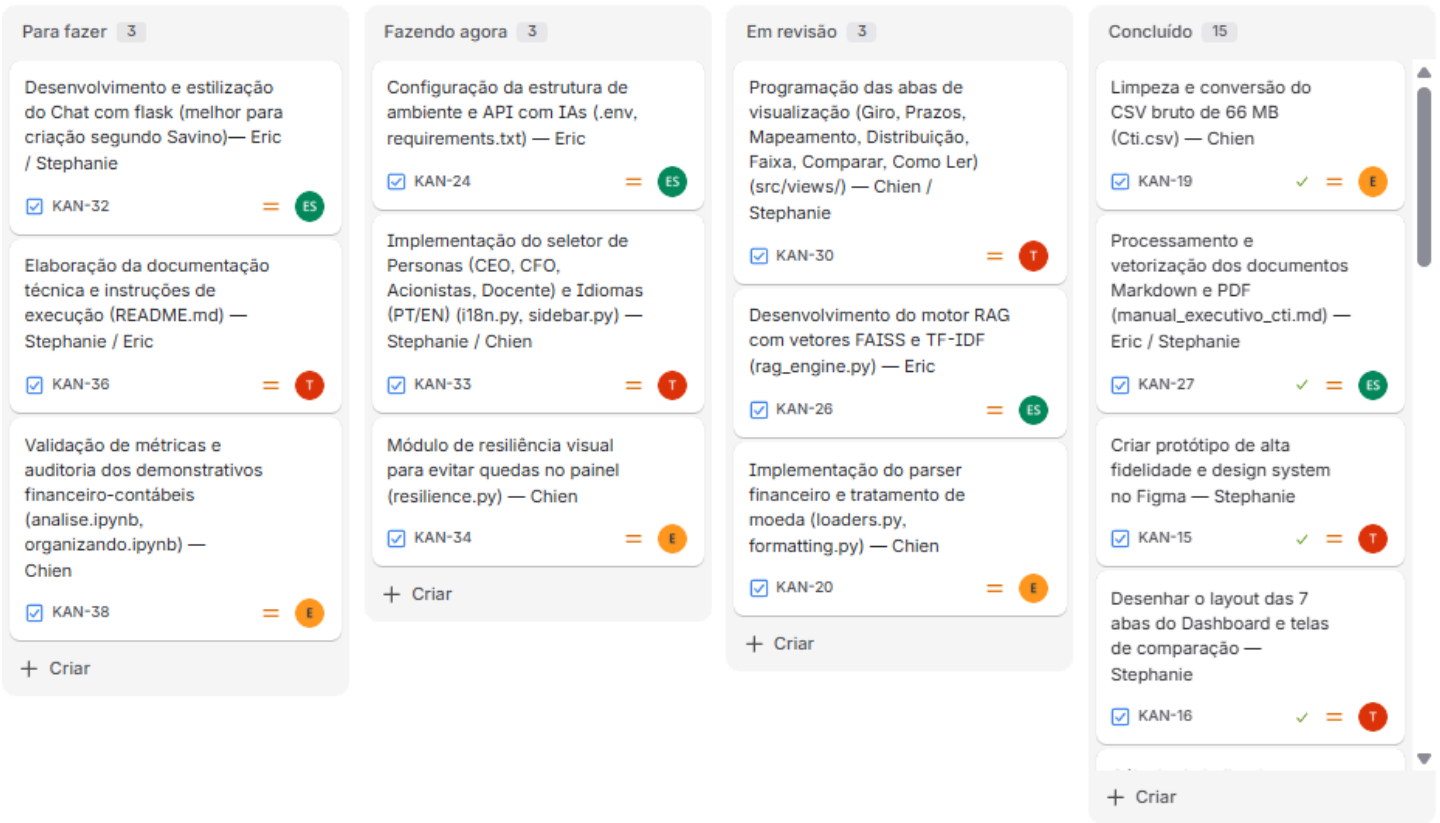
Metodologia Ágil Aplicada:

Para a organização e acompanhamento das entregas do PI, nós adotamos em metodologia **Kanban**, utilizando a ferramenta **Jira** para a gestão visual do fluxo de trabalho.

O quadro do projeto foi dividido em quatro etapas operacionais:

- **Para fazer:** Reúne as pendências do projeto a serem iniciadas, tais como o desenvolvimento da interface do Chat em Flask, a elaboração da documentação técnica no README.md, a validação de métricas/demonstrativos financeiros e os testes de integração frontend/backend.
- **Fazendo agora:** Centraliza as tarefas em execução ativa, incluindo a configuração de ambiente para a API de IA(para caso a propria empresa tenha sua IA padrao), a implementação do seletor de Personas/Idiomas e o desenvolvimento do módulo de resiliência visual.
- **Em revisão:** Reservado para a fase de testes e validação de componentes desenvolvidos, como a programação das abas de visualização do painel, o motor RAG e o *parser* financeiro com tratamento de moeda.
- **Concluído:** Regista as entregas totalmente finalizadas e homologadas pela equipa, abrangendo a limpeza do ficheiro CSV, a vetorização de documentos de apoio, o protótipo no Figma e o

design do layout das abas do Dashboard.



Legenda: Quadro Kanban de acompanhamento do projeto no Jira.

Requisito Funcional (RF) e Requisito Não Funcional (RNF)

- O que é?

Requisito Funcional (RF)	Requisito Não Funcional (RNF)
Refere-se ao comportamento ativo do sistema. Define as tarefas específicas, operações, cálculos e funcionalidades diretas que o software deve executar para atender às necessidades do usuário.	Refere-se às qualidades e restrições técnicas da aplicação. Define os critérios de operação, limites de infraestrutura, padrões de segurança, desempenho, usabilidade e tecnologias que garantem o bom funcionamento do software.

Requisitos Funcionais (RF)

- **RF01 (Sanfona de Auditoria da Base Bruta):** O sistema deve disponibilizar um expander de auditoria para inspeção dos dados tabulares filtrados.
- **RF02 (Glossário Interativo):** O sistema deve integrar um glossário explicativo com *tooltips* (?) sobre fórmulas e termos contábeis/financeiros.
- **RF03 (Visão Personalizada por Persona):** O sistema deve adaptar a exibição de KPIs e painéis com base na persona selecionada (CEO, CFO, Acionistas, Docente).
- **RF04 (Internacionalização Multi-idioma):** O sistema deve suportar alternância dinâmica em tempo de execução entre os idiomas Português (PT) e Inglês (EN).
- **RF05 (Aba Explicativa / Como Ler):** O sistema deve oferecer uma seção didática com a explicação interpretativa dos números, métricas e selos baseada nos filtros atuais.
- **RF06 (Aba Comparação Multi-Cenário):** O sistema deve permitir a seleção de 2 a 3 cenários simultâneos para sobreposição e comparação direta de curvas e KPIs.
- **RF07 (Aba Faixa de Risco / Envelopes):** O sistema deve plotar o gráfico de envelope mostrando a faixa entre cenários (pior, mediana, melhor) ao longo do horizonte temporal.
- **RF08 (Aba Distribuição e Probabilidades):** O sistema deve exibir gráficos de caixa (*Boxplot*) e histogramas com curvas de distribuição do caixa do ano 12.
- **RF09 (Aba Mapeamento de Risco x Retorno):** O sistema deve renderizar um gráfico de dispersão (*Scatter Plot*) cruzando Risco vs. Rentabilidade, colorido pelos Selos de Negócio.
- **RF10 (Aba Prazos e Ciclo):** O sistema deve apresentar cartões com PMR, PME, PMP e Ciclo Financeiro acompanhados de gráficos de linha do ciclo no tempo.
- **RF11 (Aba Capital de Giro):** O sistema deve exibir gráficos de evolução temporal da NCG e do Saldo de Tesouraria, além de gráficos de composição de barras.
- **RF12 (Filtragem Global por Ano e Cenário):** O sistema deve permitir a seleção global de um cenário específico e/ou de um ano de corte (Ano 1 ao Ano 12 ou "Todos").
- **RF13 (Cálculo do Ativo Circulante Operacional - ACO):** O sistema deve consolidar o ACO somando as contas em módulo de Clientes (Contas a Receber), Estoques e Créditos Tributários.
- **RF14 (Cálculo do Passivo Circulante Operacional - PCO):** O sistema deve consolidar o PCO somando as contas em módulo de Fornecedores, Encargos Sociais e Tributos a Pagar.
- **RF15 (Cálculo da Necessidade de Capital de Giro - NCG):** O sistema deve calcular a $\$NCG = ACO - PCO\$$ para cada cenário e ano.
- **RF16 (Cálculo do Saldo de Tesouraria):** O sistema deve projetar o Saldo de Tesouraria calculado como a diferença entre o Disponível e Empréstimos de Curto Prazo.
- **RF17 (Cálculo de Prazos Médios):** O sistema deve calcular o Prazo Médio de Recebimento (PMR), Prazo Médio de Estoques (PME) e Prazo Médio de Pagamento (PMP) em dias.
- **RF18 (Cálculo do Ciclo Financeiro):** O sistema deve computar o Ciclo Financeiro através da relação $\$PMR + PME - PMP\$$.
- **RF19 (Métricas de Estrutura e Desempenho):** O sistema deve calcular índices de Liquidez Corrente ($\$Ativo \backslash Circulante / Passivo \backslash Circulante\$$), Rentabilidade ($\$Resultado / Receita\$$) e Risco ($\$Passivo \backslash Total / Ativo \backslash Total\$$).
- **RF20 (Probabilidade de Ruína/Caixa Negativo):** O sistema deve quantificar a porcentagem de cenários onde o caixa do último ano (caixa_ano12) encerra com saldo negativo.
- **RF21 (Métricas Estatísticas Agregadas):** O sistema deve calcular média, mediana, desvio padrão, mínimo, máximo e percentis (P5, P10, P50, P90, P95) sobre as variáveis projetadas.
- **RF22 (Classificação por Quartis e Percentis):** O sistema deve segmentar a distribuição dos 1.200 cenários em quartis de rentabilidade, liquidez, risco e percentil 75 de

investimento/disponibilidade.

- **RF23 (Atribuição Automática de Selos):** O sistema deve classificar hierarquicamente cada cenário em um dos 6 "Selos de Negócio" definidos.
- **RF24 (Identificação de Cenários Críticos):** O sistema deve rotular e isolar cenários categorizados sob perfis de alto risco ou baixa liquidez.
- **RF25 (Separação Estruturada de Demonstrativos):** O sistema deve desacoplar e disponibilizar visões isoladas do Balanço Patrimonial (BP), DRE e Demonstração do Fluxo de Caixa (DFC).
- **RF26 (Invalidação Automática de Cache):** O sistema deve comparar a data de modificação entre o CSV original e os arquivos Parquet de cache, regravando o cache automaticamente caso o CSV seja atualizado.
- **RF27 (Aceleração por Armazenamento Parquet):** O sistema deve gerar e gerenciar um cache em arquivos .parquet (cti_limpo.parquet, indicadores.parquet, ranking.parquet) para otimizar leituras subsequentes e mitigar o reprocessamento da base bruta.
- **RF28 (Normalização do Plano de Contas):** O sistema deve padronizar as strings de contas contábeis e identificar prefixos de demonstrações (BAL - para Balanço, DRE - para DRE, FLU - para DFC).
- **RF29 (Conversão de Tipos e Formatação):** O sistema deve converter campos monetários no formato brasileiro (ex: "1.205.264.965,52") para valores numéricos de ponto flutuante, eliminando inconsistências e tratando células nulas.
- **RF30 (Ingestão da Base Bruta):** O sistema deve carregar e tratar a base financeira estática em arquivo CSV (Cti.csv de 66 MB) contendo 1.200 cenários projetados ao longo de 12 anos.
- **RF31 (API REST Externa para Chat):** O sistema deve fornecer uma API desenvolvida em FastAPI expondo o endpoint POST /api/chat para interações de linguagem natural.
- **RF32 (Healthcheck da API):** O sistema deve disponibilizar o endpoint GET /api/health para monitoramento de disponibilidade da API.
- **RF33 (Widget de Chat Flutuante Injetado):** O sistema deve renderizar um chat flutuante na interface Streamlit via componente customizado HTML/JS/CSS isolado em iframe.
- **RF34 (Indexação Vetorial RAG com FAISS):** O sistema deve ler e fatiar documentos Markdown e PDF (como o manual_executivo_cti.md) e os dados de ranking em um índice vetorial FAISS.
- **RF35 (Motor de Busca Híbrido/Vetorizador Local):** O sistema deve implementar fallback de busca por similaridade utilizando TF-IDF local quando não houver embeddings de terceiros.
- **RF36 (Priorização Recurso-Aware RAG):** O sistema de busca deve aplicar ponderação suplementar de relevância a trechos extraídos do manual executivo e do resumo de cenários.
- **RF37 (Integração com LLM Gemini):** O sistema deve consultar a API do Google Gemini (gemini-2.0-flash, gemini-1.5-flash ou gemini-2.5-flash) para sintetizar respostas embasadas na documentação.
- **RF38 (Modo de Resposta Extrativa Fallback):** Caso nenhuma chave de API válida esteja configurada, o assistente deve retornar trechos extraídos da documentação em vez de falhar.
- **RF39 (Histórico de Conversa por Sessão):** O assistente deve manter histórico delimitado (até 12 mensagens) por session_id.
- **RF40 (Gestão Transparente de Chaves de API):** O sistema deve carregar automaticamente a GOOGLE_API_KEY a partir de arquivos .env na raiz ou subpastas do projeto.

Requisitos Não Funcionais (RNF)

- **RNF01 (Codificação Semântica por Cores):** Cada Selo de Negócio deve possuir um código de cor Hexadecimal exclusivo configurado centralizadamente em CORES_SELO.
- **RNF02 (Injeção de CSS para Métricas):** O sistema deve forçar a cor de texto branca em componentes st.metric através de injeção de CSS personalizado (_KPI_FORCE_CSS).
- **RNF03 (Contraste e Tema Dark):** A interface Streamlit deve ser configurada para o tema escuro (base = "dark"), garantindo contraste e legibilidade com cor de fundo #0E1117 e destaque primário #1F4E45.
- **RNF04 (Posicionamento e Z-Index da Interface):** O componente de chat flutuante deve manter posição fixa no canto inferior direito (bottom: 20px, right: 20px) com camada z-index elevada.
- **RNF05 (Isolamento de CSS no Chat Flutuante):** O widget de chat injetado via HTML (chat_widget.html) deve encapsular seus estilos dentro da div #cti-chat-widget para não interferir na estilização nativa do Streamlit.
- **RNF06 (Linguagem Principal):** O sistema deve ser desenvolvido integralmente em Python.
- **RNF07 (Framework de Dashboard):** A camada visual de relatórios deve ser construída utilizando a biblioteca Streamlit.
- **RNF08 (Arquitetura de Serviços Desacoplada):** A API de inteligência e chat (Backend/main.py) deve funcionar de forma autônoma e em processo separado do servidor do dashboard (Frontend/app.py).
- **RNF09 (Servidor ASGI para API):** O serviço do assistente virtual deve ser executado sobre o servidor ASGI Uvicorn em porta dedicada (8000).
- **RNF10 (Armazenamento de Dados Tabulares):** A manipulação estatística e financeira deve utilizar estritamente as bibliotecas Pandas, NumPy e PyArrow.
- **RNF11 (Compatibilidade com Scripts de Análise):** A estrutura de código em Frontend/src/models deve ser modularizada para permitir importação direta por Notebooks Jupyter (.ipynb).
- **RNF12 (Tratamento Resiliente de Renderização Visual):** A aplicação deve utilizar decoradores de resiliência (safe_render / @resilient_view) para capturar exceções em abas individuais sem derrubar o dashboard completo.
- **RNF13 (Isolamento de Erros da API de IA):** Falhas de conexão, limites de cota ou falta da chave da API Gemini não podem indisponibilizar as funcionalidades gráficas do painel financeiro.
- **RNF14 (Consistência Contábil de Sinais):** As funções de parsing e agregação devem garantir o tratamento adequado de valores absolutos (magnitude) para contas de balanço com convenção de débito/crédito invertida.
- **RNF15 (Degradação Graciosa do Chat):** Na ausência de suporte nativo a FAISS no ambiente de execução, o RAG deve utilizar fallback automático para busca por produto escalar com NumPy.
- **RNF16 (Tamanho do Footprint de Cache):** A geração dos três arquivos Parquet comprimidos não deve ultrapassar 10 MB no disco local.
- **RNF17 (Renderização Gráfica Leve):** A atualização dos gráficos Plotly ao alternar filtros na interface deve ser concluída em menos de 1 segundo.
- **RNF18 (Eficiência no Uso de Memória):** O backend de processamento de dados Pandas/NumPy não deve exceder 1,5 GB de consumo de RAM.
- **RNF19 (Tempo de Resposta do RAG/LLM):** As chamadas do chat via API FastAPI e Google

Gemini devem retornar a resposta final em menos de 4 segundos sob condições normais de rede.

- **RNF20 (Processamento da Carga Bruta Inicial):** O tratamento primário do arquivo CSV de 66 MB (primeira execução ou rebuild de cache) não deve exceder 15 segundos.
- **RNF21 (Tempo de Carregamento por Cache Parquet):** A carga e a inicialização do dashboard a partir dos arquivos .parquet devem ocorrer em menos de 2 segundos.
- **RNF22 (Gestão de Credenciais e Segredos):** Chaves de API (GOOGLE_API_KEY) nunca devem ser hardcoded nos scripts, devendo ser lidas estritamente a partir de variáveis de ambiente ou arquivos .env.
- **RNF23 (Proteção contra Versionamento Acidental):** O repositório deve manter regras em .gitignore para impedir o envio acidental de arquivos .env, bases em .parquet e diretórios .venv.
- **RNF24 (Política de CORS para API):** A API FastAPI deve configurar middleware de CORS permitindo conexões da origem da aplicação Streamlit.
- **RNF25 (Higienização de Arquivos e Textos):** O processador de documentos do RAG deve filtrar e ignorar arquivos irrelevantes, transcrições com marcação de fala informal ou executáveis acidentais na pasta de documentos.
- **RNF26 (Organização MVC/Layered):** O código frontend deve seguir a divisão em diretórios config, controllers, models e views.
- **RNF27 (Carregamento Preguiçoso de Módulos - Lazy Import):** Os pacotes src.models e src.controllers devem implementar __getattr__ para importar submódulos apenas quando forem requisitados.
- **RNF28 (Centralização da Internacionalização):** Todos os textos fixos da interface devem ser recuperados via chaves da função t() através de dicionários centralizados em config/i18n.py.
- **RNF29 (Centralização de Constantes do Sistema):** Caminhos de pasta, cores e mapeamento de contas contábeis devem ser mantidos estritamente em config/__init__.py.
- **RNF30 (Declaração Clara de Dependências):** O projeto deve manter arquivos requirements.txt independentes para as camadas de Backend e Frontend.
- **RNF31 (Execução Headless):** O servidor Streamlit deve ser configurado com headless = true para operação adequada em ambientes de servidor remoto/nuvem.
- **RNF32 (Desativação de Telemetria Externa):** O dashboard deve desativar a coleta de estatísticas de uso do Streamlit (gatherUsageStats = false).
- **RNF33 (Portas de Serviço Padronizadas):** O sistema deve operar por padrão na porta 8501 para o painel de controle e porta 8000 para a API de assistente virtual.
- **RNF34 (Persistência do Histórico de Chat):** As sessões de chat devem ser limpas automaticamente ou limitadas em memória FIFO (no máximo 12 mensagens) para evitar estouro de memória no servidor da API.
- **RNF35 (Flexibilidade de Invocação de Execução):** O ponto de entrada da aplicação deve suportar ser iniciado tanto a partir da raiz da pasta PI4 quanto de dentro do diretório Frontend.
- **RNF36 (Rastreamento por Logging de Execução):** Operações críticas de carga de arquivo, reconstrução de cache e chamadas de RAG devem emitir registros de log formatados no console.
- **RNF37 (Rigor Matemático das Contas do Balanço):** A consolidação contábil do modelo deve preservar a equação fundamental do balanço ($\text{\$Ativo} = \text{\$Passivo} + \text{\$Patrimônio Líquido}$) em todos os demonstrativos.
- **RNF38 (Padronização Numérica Regionalizada):** Formatações monetárias e de tempo devem aplicar sufixos e pontuações apropriados conforme a cultura do idioma selecionado (ex: "R\$ 1,2

mi" em PT vs "\$ 1.2M" em EN).

- **RNF39 (Suporte a Dados de Longo Prazo):** A matriz temporal deve suportar projeções contínuas de 1 até 12 anos.
- **RNF40 (Integridade de Leitura Parquet):** O engine de persistência Parquet deve utilizar compressão do PyArrow para evitar perda de precisão de ponto flutuante durante operações de IO.

Resultados Aplicados ao Projeto (Mapeamento E/P/S)

O mapeamento de Entradas, **Processamento e Saídas (E/P/S)** descreve o fluxo completo dos dados dentro do sistema. Ele detalha como os dados brutos de simulação entram na aplicação (*Entradas*), a lógica de negócios e cálculos estatísticos aplicados sobre eles (*Processamento*) e, por fim, como essas informações são apresentadas na interface visual para apoio à tomada de decisão dos gestores (*Saídas*)

Entradas (Inputs)	Processamento (Processing)	Saídas (Outputs)
Bases Brutas: Arquivos CSV com os 1.200 cenários simulados (Cti.csv) contendo variáveis contábeis/financeiras ao longo de 12 anos.	Higienização: Leitura, tratamento de dados nulos e padronização de tipos de dados.	Métricas e KPIs: Cards de resumo executivo com Probabilidade de Ruína (caixa negativo no Ano 12).
Filtros e Interação: Seleção de visões por <i>persona</i> (CFO, Acionistas, Credores), busca por cenários e seletores de camadas visuais.	Cálculos Estatísticos: Apuração de percentis (P5, P25, P50, P75, P95), médias e desvio-padrão.	Visualizações Interativas: Envelopes de probabilidade, Histograma de Liquidez, Boxplots e Matriz Risco x Retorno.
	Categorização (Selos): Agrupamento dos cenários por perfil de risco/retorno (ex: <i>Alta Rentabilidade & Alta Liquidez, Caixa Insuficiente</i>).	Comparador: Painel lado a lado de cenários para análise rápida.

Observações Reflexivas do Grupo

A construção do projeto nos permitiu comparar o que tínhamos planejado com o que realmente funcionou na prática:

- **O Problema Inicial:** Nosso código começou como um arquivo único e gigante (mais de 1.000 linhas). Além disso, os gráficos tinham erros visuais e nomes confusos, o que dificultava a leitura rápida por parte da diretoria.
- **A Solução Técnica:** Com as práticas de Engenharia de Software, decidimos quebrar esse arquivo único em partes menores e independentes (src/data, src/components, src/views). Se um gráfico falhar, o resto do painel continua funcionando.
- **O Impacto no Negócio:** Ao adicionar ferramentas como o comparador de cenários, os histogramas e a separação por perfis de risco (selos), o dashboard deixou de ser apenas um amontoado de dados e passou a responder diretamente às perguntas dos acionistas e dos credores.